

TECNOLÓGICO NACIONAL DE MÉXICO

Gestión de Proyectos

DISEÑO DE PROTOCOLO Y PAYLOAD JSON

Equipo 3A

Bolaños García Omar Gadiel

Diego Emilio Ortiz Vilchez

Perez Manriques Jorge Luis

Chavez Vega Emilio Sebastian

ING. Sistemas Computacionales | Celaya

Proyecto: Monitor de Nivel de Agua en Tinaco

KR 3.2: Análisis de Protocolo y Payload JSON

Entregable: Documento de especificación de interfaz (API Contract)

Sprint 3 | Fecha: 13 may 2026 | **Estado:** In Review

1. Objetivo

Definir la estructura exacta y optimizada del JSON que enviará el microcontrolador ESP32 hacia Firebase Realtime Database. El objetivo principal es minimizar el consumo de ancho de banda y reducir la latencia de transmisión a través del protocolo HTTP REST previamente seleccionado. Se establece el contrato de interfaz (API Contract) que estandarizará la comunicación entre el hardware, el backend en la nube y la aplicación móvil.

2. Justificación de la Optimización

El sistema realiza mediciones y transmisiones en ciclos de 500 ms. Esto equivale a 2 mensajes por segundo, 120 por minuto, y **172,800 peticiones al día**.

Debido a que el protocolo de comunicación principal es HTTP REST (el cual demostró ser el cuello de botella de la ruta E2E con una latencia media de 155 ms), reducir el tamaño del payload (cuerpo del mensaje) es fundamental para:

- Minimizar el tiempo de transmisión (Hop 2: ESP32 → WiFi).
- Reducir el costo operativo asociado a la cuota de ancho de banda de Firebase (Hop 3).
- Reducir el uso de memoria RAM (heap) en el ESP32 al serializar la cadena JSON.

3. Especificación de la Interfaz (API Contract)

- **Protocolo Base:** HTTP REST sobre TLS (HTTPS)
- **Endpoint de destino:** `PUT https://<project-id>.firebaseio.com/tinaco/lectura.json`
- **Método HTTP:** `PUT` (Sobrescribe el nodo existente para mantener solo la lectura más reciente y evitar un crecimiento descontrolado de la base de datos).
- **Headers obligatorios:**
 - `Content-Type: application/json`
 - `Connection: keep-alive` (Crucial para evitar los picos de reconexión TLS de hasta 305 ms detectados en el KR 2.2).

4. Estructura del Payload JSON

Para minimizar los bytes transmitidos, se reemplazan las claves descriptivas propuestas en KRs anteriores (como `dist_cm` o `nivel_pct`) por identificadores de un solo carácter. Además, los valores de punto flotante se truncarán a un (1) decimal de precisión, dado que el sensor HC-SR04 tiene un error máximo calibrado de 0.213 cm, haciendo irrelevantes los decimales adicionales.

4.1 Diccionario de Datos

Clave Corta	Clave Original	Tipo de Dato	Rango / Formato	Descripción
<code>d</code>	<code>dist_cm</code>	Float	10.0 a 50.0	Distancia cruda filtrada por Kalman. 1 decimal.
<code>n</code>	<code>nivel_cm</code>	Float	10.0 a 50.0	Nivel de agua en centímetros. 1 decimal.
<code>p</code>	<code>nivel_pct</code>	Float	0.0 a 100.0	Porcentaje de llenado del tinaco. 1 decimal. Usado por la app y Cloud Function.
<code>a</code>	<code>alerta</code>	Integer (Bool)	0 o 1	Flag local de alerta (0 = false, 1 = true). Permite ahorrar 4 bytes por mensaje.
<code>t</code>	<code>ts_esp32</code>	String	ISO 8601	Marca de tiempo NTP exacta (ej. <code>2026-05-13T08:58:32-06:00</code>).
<code>s</code>	<code>ts_server</code>	Object	<code>{".sv": "timestamp"}</code>	Macro de Firebase para registrar el timestamp del servidor a su llegada.

4.2 Ejemplo del Payload Optimizado a Enviar

```
{
  "d": 29.9,
  "n": 30.1,
  "p": 50.1,
  "a": 0,
  "t": "2026-05-13T08:58:32-06:00",
  "s": {".sv": "timestamp"}
}
```

5. Análisis Comparativo de Ancho de Banda

Se contrastó el payload no optimizado definido en la fase inicial del proyecto con el nuevo payload estandarizado en este KR.

Métrica	Payload Original (KR 2.1)	Payload Optimizado (KR 3.2)	Mejora
Longitud aproximada	145 bytes	95 bytes	-34%
Tráfico por hora (7,200 msjs)	~1,044 KB	~684 KB	-34%
Tráfico diario (172,800 msjs)	25.05 MB / día	16.41 MB / día	Ahorro 8.64 MB/día
Tráfico mensual estimado	751.5 MB / mes	492.3 MB / mes	Ahorro ~259 MB/ mes

6. Consideraciones de Implementación en Firmware

Para implementar este contrato en el código fuente (ESP32 C++), se deben seguir estas directrices:

- Formateo de Cadenas:** Utilizar `sprintf` o la concatenación nativa de `String` asegurando el formato `%.1f` para los valores numéricos.
- Sincronización NTP:** La clave `t` debe generarse estrictamente mediante `getISO8601()` con offset CST (`-06:00`).
- Timestamp de Servidor:** Aunque el ESP32 mande el string literal `"{ ".sv": "timestamp" }"`, Firebase inyectará el tiempo real. Vital para calcular la latencia E2E real ($T_4 - T_3$).

7. Conclusiones

- La refactorización logra una **reducción del 34% en el tamaño del payload**, pasando de ~145 a ~95 bytes por envío.
- Ahorro de **259 MB mensuales** en la capa gratuita de Firebase, asegurando la viabilidad del prototipo a largo plazo.
- El uso de enteros (0/1) en lugar de booleanos y la reducción a un solo decimal no afectan la precisión ni el sistema de notificaciones.
- El contrato soporta la arquitectura de doble timestamp necesaria para el monitoreo de latencia E2E.